[GPU 똑똑하게 쓰기 (3)] 마지막 퍼즐, 남는 GPU를 만들고 채우다 - Bin Packing과 Batch Inference 클라우드 2026. 10. 28. 16:00

https://blog.naver.com/post6155/224451378875



> 클러스터 곳곳에 GPU가 비어 있어도, 정작 대규모 학습 작업은 들어설 자리를 찾지 못합니다. 서비스가 한가한 시간에는 GPU가 남지만, 서두를 필요 없는 대량의 작업은 바쁜 시간대의 요청 사이에서 차례를 기다립니다. 흩어져 있는 빈 GPU를 한 서버로 몰아 큰 자리를 만드는 Bin Packing과, 급하지 않은 추론 요청을 모아 여유 시간대에 처리하는 Batch Inference를 살펴봅니다.

핵심 인사이트

- **남은 GPU는 '몇 장'보다 '어디에'가 중요합니다:** 서빙에 쓰이는 GPU가 노드마다 흩어지면, 빈 GPU를 다 합치면 충분해도 대규모 학습을 올릴 수 없습니다.
- **채울 때도, 비울 때도 순서가 있습니다:** 확장할 때는 GPU를 많이 쓰는 노드부터 채우고, 축소할 때는 그 역순으로 비워 빈 노드를 통째로 확보합니다.
- **급하지 않은 추론은 여유 시간대에 처리합니다:** Batch Inference는 즉시 응답이 필요 없는 요청을 모아 공용 Pool의 남는 처리량으로 처리합니다.
- **실시간 추론을 먼저 처리하고, 남는 자원을 나눠 씁니다:** 배치 추론은 실시간 추론이 쓰고 남은 처리 여유를, Spot 학습은 비워진 노드를 채우다가, 실시간 추론에 자원이 필요해지면 자리를 내줍니다.

남는 GPU는 저절로 생기지 않는다

GPU를 더 똑똑하게 쓰는 대표적인 방법으로는, 실시간 추론 GPU를 한가한 시간에 학습에 빌려주고 필요할 때 돌려받는 Mixed Workload[1]와, 기종과 용도에 따라 나뉜 클러스터를 하나의 자원 Pool처럼 쓰는 멀티 클러스터링과 Failover[2]가 있습니다.

그런데 두 기술 모두 한 가지를 전제로 합니다. 빌려줄 GPU가 '한 노드에 통째로' 비어 있어야 한다는 점입니다. 클러스터 사이의 배치를 아무리 잘 정리해도, 클러스터 안에서 서빙 인스턴스(GPU에 모델을 올려 요청을 처리하는 단위)가 늘었다 줄었다 할 때마다 빈자리가 노드 곳곳으로 흩어진다면 학습은 여전히 들어설 곳을 찾지 못합니다. 자리를 모았다 하더라도 그 시간을 채울 수요가 학습뿐이라면, 학습 요청이 없는 시간의 GPU는 다시 놀게 됩니다.

공장에 비유하면 이렇습니다. 빈 작업대가 공장 곳곳에 한두 개씩 흩어져 있으면 넓은 작업대가 필요한 큰 주문은 받을 수 없습니다. 빈 작업대를 한쪽으로 모아야 큰 주문을 받을 수 있고, 급하지 않은 주문은 모아 두었다가 주간 라인이 한가해지는 야간조에 넘겨야 공장이 쉬지 않습니다.

이번 글에서는 흩어져 있는 빈 GPU를 한 서버로 몰아 큰 자리를 만드는 Bin Packing과, 급하지 않은 추론 요청을 모아 여유 시간대에 처리하는 Batch Inference를 차례로 살펴봅니다. 그리고 이 두 가지가 실시간 추론, Spot 학습과 함께 하나의 GPU Pool을 나눠 쓰는 구조를 정리하며 GPUaaS Tech 시리즈를 마무리합니다.

유휴 GPU는 충분한데, 대규모 학습은 왜 대기할까?

기본 분산 배치가 만드는 GPU 단편화



[그림1] 분산 배치와 집중 배치의 유휴 GPU 비교 (출처: 삼성SDS)

[그림1]을 먼저 보겠습니다. GPU 8장이 장착된 서버, 즉 노드 4대로 구성된 클러스터에 모두 32장의 GPU가 있습니다. GPU 1장을 사용하는 서빙 인스턴스가 24개 떠 있다면, 클러스터에는 GPU 8장이 남습니다. 숫자만 보면 GPU 8장이 필요한 학습 작업 하나를 올리기에 충분합니다.

그런데 왼쪽처럼 인스턴스가 노드마다 6개씩 고르게 흩어져 있으면, 빈 GPU도 노드마다 2장씩 흩어집니다. 대규모 학습은 한 노드 안의 GPU 8장을 통째로 써야 제 속도가 납니다(노드 안의 GPU끼리는 NVLink라는 고속 연결로 묶여 있기 때문입니다). 그래서 빈 GPU가 8장이나 있는데도 학습은 들어갈 노드를 찾지 못하고 기다립니다. 이처럼 빈 GPU의 총량은 충분한데 여러 노드에 쪼개져 있어 쓰지 못하는 상태를 GPU 단편화(Fragmentation)라고 합니다. 오른쪽처럼 같은 인스턴스 24개를 노드 3대에 몰아 두면 4번 노드가 통째로 비어 학습이 바로 시작됩니다.

이렇게 흩어지는 것은 스케줄러가 잘못 동작해서가 아닙니다. 쿠버네티스의 기본 스케줄러는 새 Pod를 배치할 때 자원 여유가 많은 노드에 높은 점수를 주는 방식(LeastAllocated)을 기본으로 사용합니다.[3] 부하를 고르게 나누고, 노드 한 대가 장애로 빠져도 서비스 전체가 흔들리지 않게 하려는 설계입니다. 서빙만 운영하는 클러스터라면 이것이 가용성을 지키는 합리적인 기본값입니다.

문제는 서빙과 학습이 같은 GPU를 나눠 쓰는 혼합 클러스터에서 같은 원칙이 정반대의 결과를 낸다는 점입니다. 혼합 클러스터에서 GPU 단편화를 만드는 쪽은 서빙 인스턴스입니다. 서빙은 트래픽에 따라 하루에도 여러 번 늘었다 줄었다 하기 때문에, 배치 원칙을 바꾸지 않으면 빈자리는 늘 노드 곳곳에 흩어진 채로 남습니다.

해법: 채워서 비운다, Most-Allocated Bin Packing

Bin Packing은 크기가 제각각인 물건을 가장 적은 수의 상자에 담는 고전적인 최적화 문제에서 온 이름입니다. GPU 클러스터에 옮겨 보면, 워크로드를 가능한 한 적은 수의 노드에 빽빽하게 채워 나머지 노드를 통째로 비워 두는 배치 전략입니다.

쿠버네티스는 이를 위한 설정을 이미 제공합니다. 스케줄러가 노드에 점수를 매기는 NodeResourcesFit 플러그인의 점수 전략을 MostAllocated로 바꾸면, 자원을 많이 사용하고 있는 노드일수록 높은 점수를 받습니다.[3] 이때 어떤 자원을 기준으로 채울지 가중치를 줄 수 있는데, GPU 클러스터에서는 CPU나 메모리보다 GPU 사용량에 가중치를 두어야 GPU가 많이 찬 노드부터 채워집니다.

판단 기준: 모으는 만큼 커지는 장애 영향 범위

공짜는 없습니다. 인스턴스를 적은 수의 노드에 모으면, 그 노드 한 대에 장애가 났을 때 영향을 받는 인스턴스도 많아집니다. 노드에 고르게 흩어 두는 기본 방식이 지키려던 가용성을 일부 내주는 셈입니다. 따라서 Bin Packing은 모든 서빙 워크로드에 일괄 적용하기보다 서비스의 성격에 따라 나눠 적용하는 편이 안전합니다.

오토스케일링 대상인 모델은 최소 인스턴스 수를 2개 이상으로 두어 노드 한 대의 장애가 서비스 중단으로 이어지지 않게 하고, 인스턴스 수가 고정된 모델은 기존의 분산 배치를 유지하는 식입니다. 장애가 난 노드의 인스턴스는 이상 탐지와 자동 재배치 체계가 다른 노드로 옮겨 줍니다.[2] 결국 판단 기준은 '이 서비스가 노드 한 대의 장애를 얼마나 견딜 수 있는가'이며, 그 답에 따라 모을 서비스와 흩어 둘 서비스를 구분해야 합니다.

축소(Scale-In) 때 다시 생기는 GPU 단편화

쿠버네티스는 붐비는 노드부터 지운다

늘릴 때 모아서 배치했다고 끝이 아닙니다. 트래픽이 줄어 인스턴스를 줄일 때 어떤 인스턴스부터 지우느냐에 따라, 정성껏 모아 둔 배치가 한 번에 다시 흩어질 수 있기 때문입니다.

쿠버네티스의 ReplicaSet 컨트롤러는 인스턴스 수를 줄일 때 정해진 기준에 따라 삭제할 Pod를 고릅니다. 그 중 하나가 '같은 노드에 같은 종류의 Pod가 더 많이 몰려 있는 Pod를 먼저 지운다'는 기준입니다.[4] 이 역시 분산 배치를 전제로 한 설계라서, 한 노드에 인스턴스가 몰려 있으면 그 노드의 인스턴스부터 줄여 다시 고르게 흩어 놓으려 합니다. Bin Packing과는 정확히 반대 방향입니다.

예를 들어 GPU 1장을 쓰는 인스턴스 10개가 Most-Allocated 방식으로 GPU 8장 노드 두 대에 8개와 2개로 배치되어 있다고 해 봅니다. 인스턴스를 8개로 줄일 때 우리가 원하는 결과는 두 번째 노드의 2개를 지워 그 노드를 통째로 비우는 것입니다. 하지만 기본 동작은 인스턴스가 많이 몰린 첫 번째 노드에서 먼저 지우려 하므로, 6개와 2개가 남기 쉽습니다. 남은 GPU는 8장이지만 두 노드에 6장과 2장으로 흩어져 있어, 8GPU 학습은 여전히 들어갈 수 없습니다.



[그림2] Scale-In 시 기본 삭제 순서와 역순 회수 비교 (출처: 삼성SDS)

해법: 배치의 역순으로 회수한다

빈 노드를 만들려면 마지막에 배치된 인스턴스부터 지워야 합니다. 가득 찬 노드에 먼저 들어간 인스턴스는 오래 남기고, 비어 가는 노드에 나중에 들어간 인스턴스부터 정리하는 것입니다. 상자를 채운 순서의 역순으로 꺼내는 셈입니다.

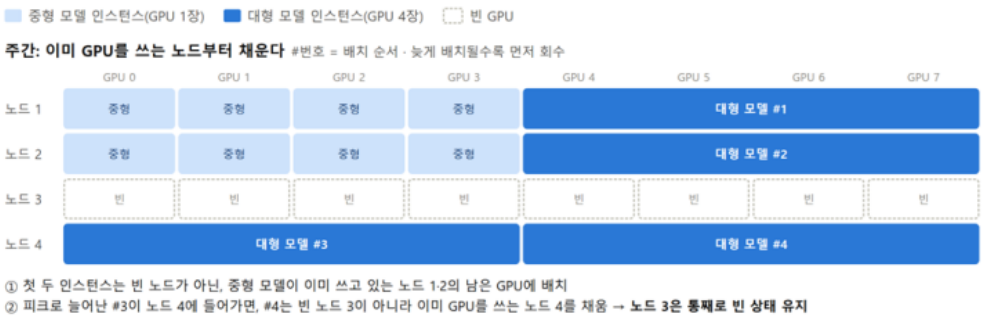
쿠버네티스는 이를 위해 Pod마다 삭제 비용(pod-deletion-cost)을 지정하는 방법을 제공하며, 비용이 적은 Pod일수록 축소 시 먼저 삭제됩니다.[4] 다만 이 값은 누군가 붙여 주어야 합니다. 그래서 서빙 인스턴스가 배치되는 순서에 따라 먼저 배치된 인스턴스에는 높은 값을, 나중에 배치된 인스턴스에는 낮은 값을 자동으로 매기도록 구성했습니다. 이렇게 하면 축소는 언제나 가장 최근에 늘어난 인스턴스부터 일어나고, 빈자리는 흩어지지 않고 한 노드에 모입니다.

확장할 때 모으는 Most-Allocated와 축소할 때 역순으로 비우는 삭제 비용 관리는 한 쌍으로 동작해야 의미가 있습니다. 어느 한쪽만으로는 서빙이 하루에도 여러 번 늘었다 줄었다 하는 동안 모아 둔 배치를 지킬 수 없기 때문입니다.

GPU가 쉬지 않는 하루: 혼합 클러스터 시나리오

두 기술이 실제로 어떻게 맞물리는지, GPU 8장을 장착한 B300 노드 4대로 구성된 혼합 클러스터의 하루를 따라가 보겠습니다. 이 클러스터에서는 인스턴스 하나에 GPU 1장을 쓰는 중형 모델과 GPU 4장을 쓰는 대형 모델이 함께 운영됩니다. 대형 모델은 요청량에 따라 인스턴스가 2개에서 6개까지 늘었다 줄었다 합니다.

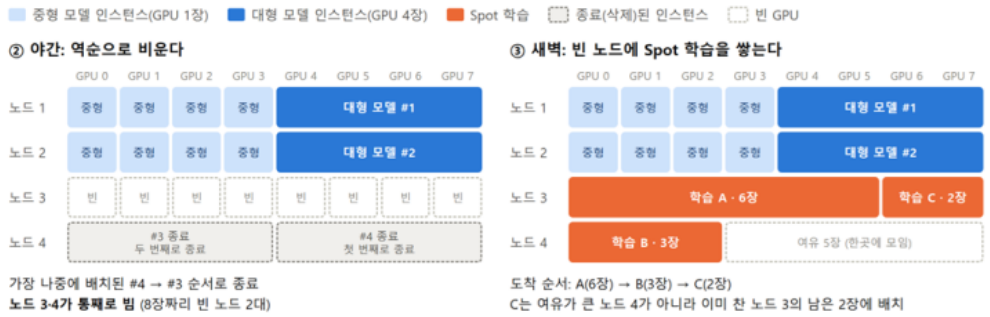
① 주간: 이미 찬 노드부터 채운다



[그림3] 주간 확장: 사용 중인 노드부터 채우기 (출처: 삼성SDS)

[그림3]은 주간에 요청이 늘면서 대형 모델이 확장되는 과정입니다. 중형 모델은 1번과 2번 노드의 GPU 4장씩을 사용합니다. 대형 모델의 첫 두 인스턴스는 Most-Allocated 원칙에 따라 이미 GPU를 쓰고 있는 1번과 2번 노드의 남은 GPU 4장에 각각 배치되어, 두 노드는 가득 찹니다. 비어 있는 3번, 4번 노드에 먼저 배치되지 않는다는 점이 핵심입니다. 오후 피크로 요청이 늘어 세 번째 인스턴스(#3)가 4번 노드에 배치되면, 네 번째 인스턴스(#4)는 빈 3번 노드가 아니라 이미 GPU를 쓰고 있는 4번 노드로 가서 그 노드를 채웁니다. 각 인스턴스에는 배치된 순서에 따라 삭제 비용이 매겨지며, 먼저 배치된 인스턴스일수록 높습니다.

② 야간: 배치의 역순으로 비운다



[그림4] 야간 축소 후 빈 노드에 Spot 학습 배치 (출처: 삼성SDS)

[그림4]는 밤사이 인스턴스가 줄어 빈 노드가 생기고, 그 자리에 Spot 학습이 채워지는 과정입니다. 밤이 되어 요청이 줄면 대형 모델의 인스턴스는 최소치인 2개로 줄어듭니다. 가장 나중에 배치되어 삭제 비용이 가장 낮은 #4가 먼저, #3이 다음으로 종료되면서 4번 노드는 통째로 비고, 3번 노드도 그대로 비어 있습니다. 클러스터에 GPU 8장짜리 빈 노드가 두 대 생긴 것입니다.

③ 새벽: 빈 노드에 Spot 학습을 쌓는다

이제 이 자리에 Spot 학습이 들어옵니다. Spot 학습 역시 같은 Most-Allocated 원칙을 따르므로, 들어오는 순서대로 이미 GPU를 쓰고 있는 노드부터 채워 나갑니다. GPU 6장을 요구하는 학습 A가 3번 노드에 배치되고, 3장을 요구하는 학습 B는 3번 노드에 자리가 2장뿐이라 4번 노드에 배치됩니다. 이어서 2장을 요구하는 학습 C는 여유가 더 많은 4번 노드가 아니라 이미 6장이 찬 3번 노드의 남은 두 자리에 들어갑니다. 3번 노드는 다시 가득 차고, 4번 노드에는 5장의 여유가 한곳에 모여 남습니다.

④ 다시 아침: 남은 자리로 늘리고, 모자라면 Spot을 회수한다



[그림5] 아침 재확장과 Spot 학습 회수 (출처: 삼성SDS)

[그림5]는 아침에 다시 확장하면서, 모자란 자리를 Spot 학습 회수로 확보하는 과정입니다. 아침이 되어 요청이 다시 늘면 대형 모델이 확장을 시작합니다. 4번 노드에 GPU 5장이 한곳에 모여 있으므로, 첫 번째 추가 인스턴스는 학습을 건드리지 않고 바로 배치됩니다. 여유를 흩어 두지 않고 모아 둔 덕분입니다.

요청이 더 늘어 인스턴스를 하나 더 늘려야 하는데 GPU 4장이 모인 노드가 없다면, 그때 Spot 학습을 회수합니다. 회수할 학습을 고를 때도 기준이 필요합니다. 3번 노드의 학습 A(6장)를 회수하면 GPU 6장이 비어 인스턴스를 올리고도 2장이 남고, 4번 노드의 학습 B(3장)를 회수하면 남아 있던 1장과 합쳐 정확히 4장이 확보됩니다. 어느 쪽을 고르느냐에 따라 멈추는 학습의 규모와 회수 후 남는 자리가 달라지므로, 서빙에 필요한 자리를 확보하면서 학습 손실이 가장 작은 쪽을 고르는 것이 원칙입니다.

회수된 학습은 끝난 것이 아니라 잠시 멈춘 것입니다. 학습을 멈추지 않고 체크포인트를 저장하는 Concurrent Checkpointing이 회수 직전까지의 진행 분을 지키고, 저장된 체크포인트를 새 GPU 구성에 맞

게 다시 나누는 Checkpoint Resharding이 다음에 확보되는 GPU에서 학습을 이어 줍니다.[1] 장애에 대응해 작업을 옮기는 Failover와 달리, 이 회수는 서빙에 자리를 내주기 위해 계획적으로 자리를 비우는 일입니다.[2]

이렇게 보면 GPU는 하루 종일 쉬지 않습니다. 낮에는 서빙이, 밤에는 학습이 같은 GPU를 씁니다. 실제로 DeepSeek도 부하가 높은 낮에는 모든 노드에 추론 서비스를 올리고, 부하가 낮은 밤에는 추론 노드를 줄여 그 자원을 연구와 학습에 배정한다고 공개했습니다.[5] 그런데 질문이 하나 남습니다. 밤에 남는 GPU를 채울 수요가 학습밖에 없을까요? 학습 요청이 없는 밤이라면, 혹은 노드를 통째로 비울 만큼 서빙이 줄지 않는 밤이라면 어떻게 해야 할까요?

비워진 시간을 채울 수요가 학습뿐일까?

지금 당장 답이 필요하지 않은 추론

흔히 떠올리는 추론은 실시간 추론입니다. 사용자가 질문하면 수 초 안에 첫 토큰이 나와야 하고, 그 기준을 지키기 위해 캐시, 라우팅, 오토스케일링 같은 기술이 동원됐습니다. 하지만 기업의 AI 활용에는 이와 성격이 다른 추론도 적지 않습니다.

쌓여 있는 고객 문의 수만 건을 유형별로 분류하는 작업, 사내 문서 전체를 요약해 검색 색인을 만드는 작업, 상품 정보 수십만 건에서 속성을 추출하는 작업, 검색 서비스를 위해 문서 임베딩을 새로 생성하는 작업, 새 모델을 도입하기 전에 평가 데이터 세트 전체를 돌려 보는 작업이 그렇습니다. 이들은 요청 하나하나의 응답 속도보다 전체 작업이 정해진 시간 안에 끝나는 것이 중요합니다. 결과가 1초 뒤에 나오든, 몇 시간 뒤에 나오든, 내일 아침 업무가 시작되기 전에만 나오면 되는 요청입니다.

실시간 API로 처리할 때 생기는 일

이런 대량 요청을 실시간 API로 처리하면 몇 가지 문제가 생깁니다. **먼저 사용량 한도입니다.** 실시간 API에는 특정 사용자의 자원 독점을 막기 위한 토큰 기반 한도가 걸려 있습니다.[6] 수십만 건의 요청을 한꺼번에 보내면 한도에 걸려 거절되고, 사용자는 요청 속도를 조절하는 로직을 직접 만들어야 합니다.

다음은 실패 처리입니다. 대량 요청을 보내다 보면 일부는 시간 초과나 일시적 오류로 실패합니다. 어떤 요청이 성공했고 어떤 요청을 다시 보내야 하는지 추적하고, 중간에 프로그램이 멈추면 어디서부터 다시 시작할지 관리하는 일이 모두 사용자의 몫이 됩니다. 정작 원했던 것은 분류 결과인데, 요청을 관리하는 프로그램을 만드는 데 더 많은 시간을 쓰게 됩니다.

마지막은 자원 경쟁입니다. 대량 요청이 업무 시간에 몰리면 같은 모델을 쓰는 실시간 요청과 같은 GPU를 두고 경쟁하게 됩니다. 급하지 않은 요청 때문에 급한 요청의 응답 속도가 느려지는 것입니다. 반대로 밤에는 실시간 요청이 줄어 GPU가 남는데도, 이 시간을 활용하려면 사용자가 따로 작업을 예약하고 실행을 지켜봐야 합니다.

맡기고 나중에 받는 Batch Inference

요청을 파일로 맡기고, 결과를 파일로 받는다

Batch Inference는 사용 방식 자체를 바꿔 이 문제를 해결합니다. 요청을 하나씩 보내고 응답을 기다리는 대신, 처리할 요청을 한 파일에 모아 한 번에 맡기고 처리가 끝나면 결과를 파일로 받는 방식입니다. 요청 속도 조절, 실패 요청 기록, 진행 관리는 서비스가 대신 맡습니다. OpenAI의 Batch API처럼 주요 LLM 서비스도 이미 같은 방식을 제공하고 있으며, 실시간 API보다 낮은 가격과 넉넉한 사용량 한도를 함께 내세웁니다.[7]

흐름은 단순합니다. 사용자는 한 줄에 요청 하나씩을 담은 JSONL(JSON Lines) 파일을 준비해 Object Storage에 올리고, 이 파일 경로를 지정해 배치 작업(Batch Job)을 생성합니다. 작업은 대기 상태로 들어갔다가 처리할 수 있는 자원이 생기는 대로 요청을 차례로 처리합니다. 성공한 요청의 응답은 결과 파일로, 실패한 요청은 실패 사유와 함께 오류 파일로 사용자의 저장소에 기록됩니다. 각 줄에는 사용자가 정한 식별자가 붙어 있어, 결과 파일의 응답이 원래 어떤 요청에 대한 것인지 쉽게 맞춰 볼 수 있습니다. 사용자는 작업의 진행 상태와 처리 건수만 확인하다가, 작업이 끝나면 결과 파일을 가져가면 됩니다.



[그림6] Batch Inference 처리 흐름 (출처: 삼성SDS)

실시간 추론과 무엇이 다른가

Batch Inference는 실시간 추론을 대체하는 기술이 아니라, 실시간 추론과 다른 약속을 맺는 방식입니다. 응답 시점을 양보하는 대신 가격과 편의를 얻습니다.

구분	실시간 추론	배치 추론
요청 방식	요청 1건마다 API 호출	요청을 모은 파일 1개로 작업 생성
응답 시점	요청 즉시	작업 완료 후
처리 우선순위	최우선	실시간 요청이 남긴 여유분으로 처리
사용량 관리	분당 토큰 한도 적용	서비스가 투입량을 조절
실패 처리	사용자가 재시도 구현	실패 요청을 오류 파일로 분리 제공
단가	표준 단가	표준 대비 할인 단가
적합한 작업	챗봇, 에이전트, 실시간 검색	대량 분류·요약·추출, 임베딩 생성, 모델 평가

[표 1] 실시간 추론과 배치 추론의 비교 (출처: 삼성SDS)

이 차이는 자원을 쓰는 방식에서도 드러납니다. 배치를 위해 별도의 GPU를 마련할 필요는 없습니다. 배치 요청은 실시간 요청과 같은 모델 인스턴스에서 처리하되, 같은 관문(Inference Gateway)을 거치면서 실시간 요청보다 낮은 우선순위를 받습니다. 실시간 요청이 남긴 처리 여유만큼만 배치가 투입되므로, 배치 작업이

늘어도 실시간 서비스의 응답 품질은 지켜집니다. 배치가 남는 자원을 쓰는 서비스라는 점이 구조 자체에 들어가 있는 것입니다.

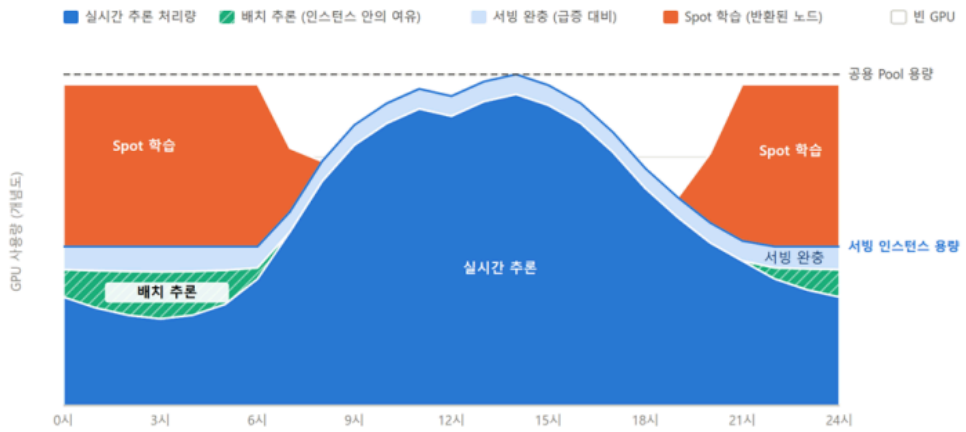
한 Pool, 세 소비자: 서빙을 중심으로 나눠 쓰기

두 종류의 여유

이제 GPU Pool을 함께 쓰는 소비자는 셋이 되었습니다. 실시간 추론, 배치 추론, Spot 학습입니다. 셋은 같은 공용 Pool을 쓰지만, 남는 자원을 채우는 단위가 다릅니다.

실시간 요청이 줄어든다고 해서 서빙 인스턴스가 곧바로 줄어드는 것은 아닙니다. 인스턴스는 그대로 떠 있지만 처리하는 요청이 적어, 인스턴스 안에 처리 여유가 생깁니다. 다만 이 여유가 하루 내내 같은 크기는 아닙니다. 피크 시간대에는 오토스케일링이 인스턴스를 수요에 맞춰 늘리기 때문에, 남는 여유는 갑작스러운 요청 증가에 대비한 완충 정도에 그쳐 배치가 들어갈 틈이 거의 없습니다. 반대로 야간처럼 수요가 최소 인스턴스 수로 감당하는 양보다 낮아지면, 인스턴스는 그대로인데 요청만 줄어 여유가 크게 넓어집니다. 배치 추론은 완충을 넘어서는 바로 이 여유를 채웁니다. 이미 모델이 올라가 있는 인스턴스에 요청을 더 흘려보내는 것이므로, 새 GPU를 확보하거나 모델을 다시 올릴 필요가 없습니다.

요청이 더 줄어 인스턴스 자체가 줄어들면, 앞에서 본 역순 회수에 따라 노드 단위로 GPU가 반환됩니다. Spot 학습은 이렇게 통째로 비워진 GPU를 채웁니다. 인스턴스 안의 여유는 배치가, 반환된 노드의 여유는 학습이 쓰는 셈입니다.



[그림7] 시간대별 공용 GPU Pool 사용 현황 (출처: 삼성SDS)

판단 기준: 서빙에 어떻게 자리를 내주는가

세 소비자가 같은 Pool을 나눠 쓰려면 기준이 필요합니다. 체크포인트와 재기동 체계가 갖춰지지 않은 환경이라면 학습을 중단시키는 손실이 커서 선점 없이 운영하는 편이 현실적입니다.[2] 반대로 Concurrent Checkpointing과 Checkpoint Resharding처럼 중단 손실을 줄이는 장치를 갖추면[1], 학습도 필요할 때 자리를 내줄 수 있는 워크로드가 됩니다. 실시간 추론이 언제나 우선이라는 점은 분명하므로, 남는 질문은 하나입니다. 서빙이 자원을 다시 필요로 할 때 배치와 학습이 각각 얼마나 빠르게, 얼마나 적은 손실로 비켜 줄 수 있는가입니다.

구분	실시간 추론	배치 추론	Spot 학습
사용하는 자원	피크 기준으로 확보한 인스턴스	인스턴스 안의 남는 처리량	축소로 반환된 노드의 GPU
서빙이 자원을 필요로 할 때	-	투입을 줄여 즉시 처리량 확보	작업을 중단하고 노드 반환
중단 시 손실	장애 시 세션과 KV 캐시 유실	처리 중이던 요청의 재처리	마지막 체크포인트 이후 진행 분
손실을 줄이는 장치	이상 탐지와 재배치	요청 단위 처리, 오류 파일	Concurrent Checkpointing, Resharding

[표 2] 공용 Pool을 함께 쓰는 세 워크로드와 서빙에 자리를 내주는 방식 (출처: 삼성SDS)

배치 추론은 이미 떠 있는 인스턴스에 요청을 흘려보내는 방식이라, 새 GPU를 확보하거나 모델을 올리는 준비 없이 바로 들어갑니다. 비킬 때도 빠릅니다. 요청 단위로 처리되기 때문입니다. 실시간 요청이 갑자기 늘면 배치 요청의 투입만 줄이면 되고, 처리 중이던 요청 몇 건 외에는 잃는 것이 없습니다.

Spot 학습은 서빙 인스턴스가 줄어 노드가 통째로 비었을 때 비로소 들어가고, 서빙이 다시 노드를 필요로 할 때 자리를 내줍니다. 이때 마지막 체크포인트 이후의 진행 분은 잃지만, Concurrent Checkpointing이 그 폭을 줄이고 Checkpoint Resharding이 달라진 GPU 구성에서도 학습을 이어 줍니다.

배치도 여유를 모두 가져가서는 안 된다
배치 요청이 서빙 인스턴스의 여유를 계속 채우면 인스턴스는 바빠 보이고, 오토스케일러는 인스턴스를 줄일 이유를 찾지 못할 수 있습니다. 인스턴스가 줄지 않으면 노드가 비지 않고, 노드가 비지 않으면 Spot 학습이 들어갈 자리도 생기지 않습니다. 배치가 여유를 채우는 만큼 학습의 몫이 줄어드는 구조입니다.

그래서 배치에는 우선순위와 함께 유량 제어가 필요합니다. 실시간 요청의 여유와 대기 중인 학습 수요를 함께 보면서, 배치 요청을 얼마나 흘려보낼지 조절하는 것입니다. 서빙과 배치, 학습 사이의 배분은 하나의 고정된 규칙보다 상황에 따라 조정할 수 있는 구조가 중요합니다.

그리고 이 구조의 바닥에는 Bin Packing이 있습니다. 서빙 인스턴스를 모아 두고 역순으로 비울 수 있어야 반환되는 GPU가 노드 단위로 모이고, 그래야 학습이 그 자리를 쓸 수 있습니다. 배치 추론이 인스턴스 안의 시간을 채우고 Bin Packing이 노드 단위의 공간을 만들어 낼 때, 공용 Pool의 GPU는 비로소 하루 종일 일하게 됩니다.

이 기술들을 실제 GPU 운영에 어떻게 적용할까?

지금까지 살펴본 기술이 실제 서비스에 적용된 사례 중 하나가 Samsung Cloud Platform의 GPU 서비스입니다. 서빙과 Spot 학습이 함께 운영되는 혼합 클러스터에서는 앞서 살펴본 원칙을 바탕으로, 워크로드를 GPU가 찬 노드부터 채우고 축소할 때는 배치의 역순으로 정리해 빈 노드를 확보하는 방향으로 운영하고 있습

니다. 다만 모든 서비스에 같은 방식을 일괄 적용하지는 않습니다. 서비스가 노드 장애를 견딜 수 있는 정도와 트래픽 변화, 학습 수요 같은 운영 상황을 함께 고려해 적용 범위를 유연하게 조정하고 있습니다.

Batch Inference는 2026년 12월 Simple AI Inference 서비스에서 제공될 예정이며, 요청 파일과 결과·오류 파일은 모두 사용자가 지정한 Object Storage 경로에 머물러, 모델의 입력과 출력이 서비스 저장소에 따로 남지 않습니다. 한 파일 안에서도 요청마다 모델을 지정할 수 있어 요약과 임베딩 생성처럼 성격이 다른 요청을 함께 맡길 수 있고, 작업은 콘솔과 OpenAPI 모두에서 생성하고 확인할 수 있습니다.

Bin Packing과 Batch Inference는 따로 도입할 수도 있지만, 함께 갖출 때 효과가 커집니다. 자체 GPU 인프라를 운영하고 있거나 GPU 서비스를 검토하고 있다면, 다음 질문으로 현재 상태를 점검해 볼 수 있습니다.

- 서빙 워크로드별로 노드 집중 배치를 허용할 수 있는지 구분하고, 노드 한 대의 장애를 견딜 최소 인스턴스 수를 정했는가?
- 인스턴스를 줄일 때 어떤 인스턴스부터 제거되는지 제어하고 있는가?
- 전체 추론 요청 중 즉시 응답이 필요 없는 요청이 얼마나 되는지 파악했는가?
- 배치 작업의 완료 기한이 실제 업무 일정(야간 정산, 익일 리포트 등)과 맞는가?
- 배치 결과에서 실패한 요청을 확인하고 다시 제출하는 절차가 마련되어 있는가?
- 실시간 추론, 배치 추론, 학습 사이의 우선순위와 자원 배분 원칙이 조직 안에서 합의되어 있는가?

마치며

남는 GPU는 저절로 생기지 않습니다. 서빙 인스턴스를 찬 노드부터 채우고 그 역순으로 비워야 학습이 들어갈 빈 노드가 만들어지고, 급하지 않은 추론을 남는 시간으로 옮겨야 인스턴스 안의 여유가 채워집니다. Bin Packing은 흩어진 빈 GPU를 한곳에 모으고, Batch Inference는 그 시간을 채울 수요를 만듭니다.

이번 글로 GPUaaS Tech 시리즈를 마칩니다. 여덟 편의 글은 서로 다른 기술을 다뤘지만, 돌아보면 모두 같은 질문에 대한 서로 다른 답이었습니다. 확보한 GPU 한 장이 하루 동안 얼마나 쓸모 있는 일을 하게 만들 것인가라는 질문입니다. <표3>은 각 편이 답한 질문을 고객의 관점에서 다시 정리한 것입니다. 궁금한 질문이 있다면 해당 아티클을 함께 읽어 보셔도 좋습니다.

고객이 자주 묻는 질문	관련 아티클	살펴볼 내용
GPU를 직접 구축할까, 서비스로 이용할까?	1편 GPU는 필수, 구축은 부담	구축과 서비스 이용 시 고려할 운영 범위
우리 작업에는 어떤 GPU가 맞을까?	2편 GPU 사양보다 워크로드 정합성이 먼저다	모델/작업 조건과 GPU 구성의 관계
같은 GPU로 추론을 더 처리할 수 있을까?	3편 토큰 팩토리의 생산성을 끌어올리는 인퍼런스 기술	캐시, 스케줄링, 오토스케일링의 역할
어떤 요청을 받고, 어디로 보낼까?	4편 AI 추론 서비스의 새로운 관문	사용량 통제와 서버 선택 방식

GPU 메모리가 먼저 차면 어떻게 할까?	5편 GPU 메모리 병목을 해결하는 세 가지 방법	병목 원인에 따른 접근과 제약
남는 서빙 GPU를 학습에 쓸 수 있을까?	6편 모델 학습은 더 쉽게, GPU 활용은 더 효율적으로	자원 회수와 학습 재개 조건
GPU가 흩어지고 장애가 나면?	7편 알아서 배치하고, 장애에도 이어간다	클러스터 간 배치와 장애 복구
남는 GPU를 학습·배치에 어떻게 쓸까?	8편 마지막 퍼즐, 남는 GPU를 만들고 채우다	빈 노드 확보와 유휴 처리량 활용

[표 3] GPU 운영에서 마주치는 질문들 (출처: 삼성SDS)

지금까지 SCP GPUaaS 서비스에 적용되어 있는 GPU 인프라부터 추론/학습 서비스의 다양한 기술들을 알아보았으며, 표의 질문을 위에서 아래로 따라가면 관심이 옮겨 간 흐름이 보입니다. 처음에는 어떤 GPU를 확보할지를 물었고, 다음에는 확보한 GPU에서 얼마나 많은 토큰을 만들지를, 그다음에는 그 GPU를 서빙과 학습, 배치가 어떻게 나눠 쓸지를 물었습니다. AI 인프라의 경쟁력이 GPU를 얼마나 확보했는가에서 확보한 GPU를 얼마나 잘 운영하는가로 옮겨 가고 있다는 뜻입니다.

GPU 한 장의 가치는 그 GPU가 얼마나 비싼가가 아니라, 하루 24시간 동안 얼마나 쓸모 있는 일을 했는가로 결정됩니다. 낮에는 실시간 추론이, 남는 처리량은 배치 추론이, 비워진 노드는 학습이 채우고, 장애가 나도 작업이 이어지는 구조. SCP GPUaaS가 지향해 온 것은 고객이 비용을 낸 GPU 시간이 빠짐없이 일하는 인프라입니다. 그동안 시리즈를 함께 읽어 주신 독자 여러분께 감사드립니다.

FAQ

Q1. Bin Packing을 적용하면 서비스 가용성이 떨어지지 않습니까?
A. 인스턴스를 적은 수의 노드에 모으므로 노드 한 대의 장애가 영향을 주는 범위는 커집니다. 그래서 오토스케일링 대상 모델은 최소 인스턴스 수를 2개 이상으로 유지하고, 장애 노드의 인스턴스를 자동으로 재배치하는 체계와 함께 운영해야 합니다. 노드 장애를 견디기 어려운 서비스는 기존의 분산 배치를 유지하는 것이 안전합니다.

Q2. 인스턴스를 줄이는 순서가 왜 중요합니까?
A. 쿠버네티스의 기본 동작은 인스턴스가 많이 몰린 노드의 Pod부터 삭제해 배치를 다시 고르게 만듭니다. 이 경우 확장할 때 모아 둔 배치가 축소 한 번에 흩어져, 남은 GPU의 총량은 같아도 대규모 학습 작업이 들어갈 빈 노드가 생기지 않습니다. 마지막에 배치된 인스턴스부터 삭제되도록 순서를 관리해야 빈자리가 한 노드에 모입니다.

Q3. Batch Inference와 실시간 API는 무엇이 다릅니까?
A. 실시간 API는 요청 하나에 즉시 응답하는 방식이고, Batch Inference는 요청을 모은 파일을 한 번에 맡기고 처리가 끝나면 결과 파일로 받는 방식입니다. 응답 시점을 양보하는 대신 사용량 한도와 재시도 관리의 부담이 줄고, 실시간보다 낮은 단가로 이용할 수 있습니다.

Q4. 배치 작업은 얼마나 빨리 끝납니까?
A. 배치 요청은 실시간 요청이 남긴 여유 자원으로 처리되므로, 완료 시간은 그 시점의 실시간 부하에 따라 달라집니다. 완료 시점이 중요한 작업이라면 여유 있게 제출하고, 정해진 시간 안에 반드시 응답이 필요한 요청은 실시간 API로 처리하는 것이 적합합니다.

Q5. 배치 추론과 Spot 학습은 남는 GPU를 어떻게 나눠 씁니까?

A. 배치 추론은 떠 있는 서빙 인스턴스 안의 남는 처리량을, Spot 학습은 서빙 인스턴스가 줄어들면서 반환된 노드 단위의 GPU를 사용합니다. 회수 비용이 낮은 배치 추론이 우선하지만, 배치가 여유를 모두 차지하면 인스턴스가 줄지 않아 학습의 자리가 생기지 않으므로 배치 투입량을 조절하는 유량 제어가 함께 필요합니다.

References

[1] Samsung SDS. [GPU 똑똑하게 쓰기 (1)] 모델 학습은 더 쉽게, GPU 활용은 더 효율적으로 - Managed Training과 Mixed Workload. https://www.samsungsds.com/kr/insights/managed-training-mixed-workload.html
[2] Samsung SDS. [GPU 똑똑하게 쓰기 (2)] 알아서 배치하고, 장애에도 이어간다 — Multi-Clustering과 GPU Failover. (게재 후 링크 기입)
[3] Kubernetes. Resource Bin Packing. https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/
[4] Kubernetes. ReplicaSet – Pod deletion cost. https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/#pod-deletion-cost
[5] DeepSeek. Day 6: One More Thing, DeepSeek-V3/R1 Inference System Overview. https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md
[6] Samsung SDS. AI 추론 서비스의 새로운 관문, Inference Gateway. https://www.samsungsds.com/kr/insights/inference-gateway-llm-serving.html
[7] OpenAI. Batch API. https://developers.openai.com/api/docs/guides/batch

▶ 이 아티클은 AI를 활용하여 제작되었습니다.
원고 초안 작성 및 톤앤 매너 보정 : Claude